Java Backend Architecture

Interview Series

Chapter 15.4
Event-Driven Architecture

50+ Interview Q&A | Code Examples | Architecture Diagrams | Advanced Topics

Chapter 15.4 – Event-Driven Architecture

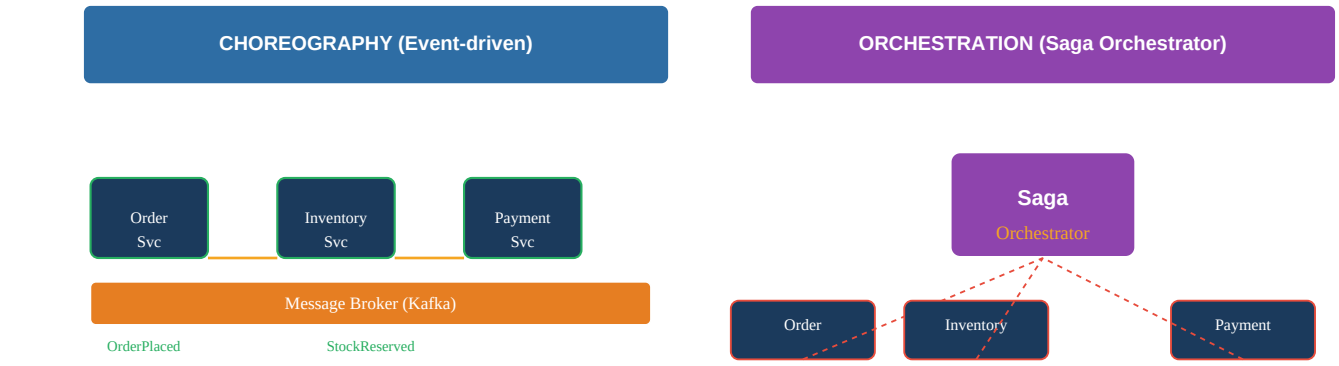
Introduction

Event-Driven Architecture (EDA) decouples services by communicating through events rather than direct calls. Producers publish facts about what happened; consumers react independently. This enables loose coupling, independent scaling, temporal decoupling (consumer need not be running when producer sends), and natural audit trails. The key design challenges: event schema evolution, ordering guarantees, idempotency, and the outbox pattern to guarantee at-least-once delivery.

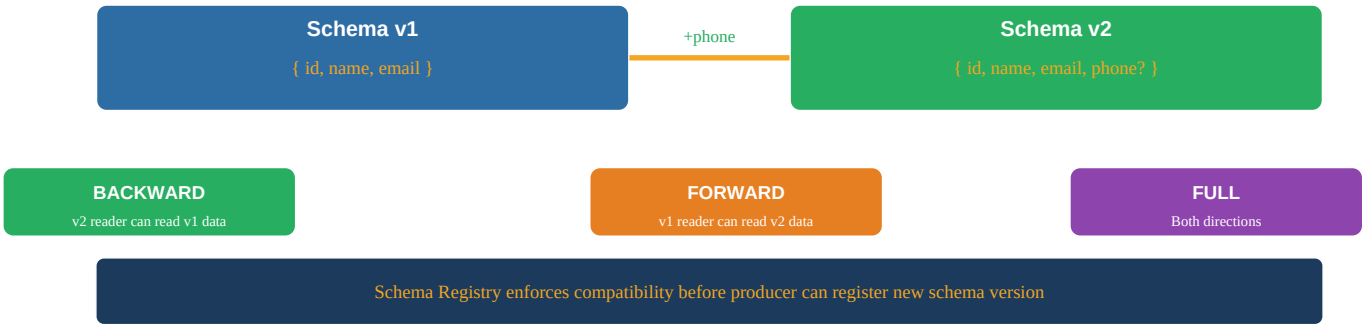
Key Definitions

Notification Event	Minimal event signaling something happened—consumer must call back for details. Low payload. Risk: extra round-trip. E.g., { "type": "ORDER_PLACED", "orderId": "123" }.
Event-Carried State Transfer	Event contains all data consumers need—no callback required. E.g., full Order object in event. Reduces coupling but increases payload size.
Domain Event	Event that records something meaningful in the domain. Used within a bounded context. Past tense. E.g., OrderConfirmed, PaymentProcessed.
Integration Event	Event published to other bounded contexts via message broker. Has stable schema (Published Language). Versioned and backward-compatible.
CloudEvents	CNCF standard spec for event metadata: specversion, id, source, type, time, datacontenttype. Ensures interoperability between systems.
Schema Registry	Centralized store for Avro/Protobuf schemas. Enforces compatibility (BACKWARD/FORWARD/FULL) before new schema version can be registered.
Outbox Pattern	Write event to an outbox table in the SAME transaction as business data. A relay process (Debezium CDC) reads committed outbox records and publishes to Kafka. Guarantees atomicity.
Choreography	Services react to events from the broker independently. No central coordinator. Loose coupling. Hard to trace end-to-end flow. Works for simple flows.
Orchestration	A central Saga Orchestrator sends commands to services, receives replies, manages saga state machine. Easy to trace. Single coordination point.
Idempotent Consumer	Consumer that can safely process the same event multiple times. Required for at-least-once delivery. Track processed event IDs in a deduplication store.
Event Sourcing	Store all state changes as immutable events (event log) rather than current state. Current state is computed by replaying events. Enables full audit trail and temporal queries.
Dead Letter Queue	Topic/queue where failed messages are routed after exhausting retries. Prevents poison pill messages from blocking processing. Requires manual inspection and replay.

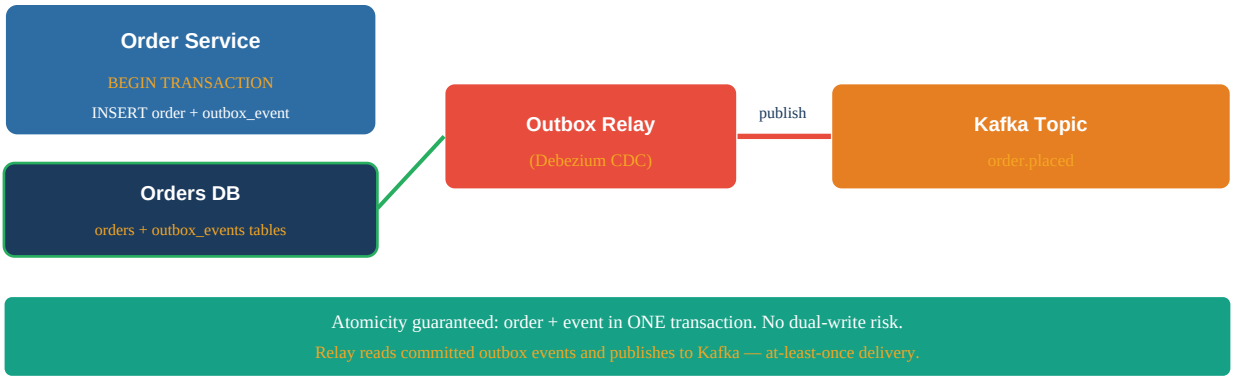
Choreography vs Orchestration Saga



Schema Evolution – Backward/Forward Compatibility



Outbox Pattern – Guaranteed Event Publishing



Event Types Comparison

Event Type	Payload	Consumer Action	Pros	Cons
Notification	Minimal (ID only)	Call back for data	Small payload	Extra round-trip
State Transfer	Full object state	Process directly	No callback needed	Large payload, coupling
Domain Event	Domain-specific data	React in context	Expressive, DDD-aligned	Context-specific
Integration Event	Stable, versioned	Cross-service react	Stable contract	Schema mgmt overhead

Spring Kafka + CloudEvents

```
// CloudEvents format for order placed event { "specversion": "1.0", "type":
"com.example.order.placed", "source": "/order-service", "id": "uuid-1234", "time":
"2025-01-01T10:00:00Z", "datacontenttype": "application/json", "data": { "orderId":
"order-123", "customerId": "cust-456", "totalAmount": {"amount": 99.99, "currency": "USD"} } }
// Producer @Service public class OrderEventPublisher { @Autowired KafkaTemplate<String,
String> kafkaTemplate; public void publishOrderPlaced(Order order) { CloudEvent event =
CloudEventBuilder.v1() .withId(UUID.randomUUID().toString())
.withType("com.example.order.placed") .withSource(URI.create("/order-service"))
.withData("application/json", orderToJson(order)) .build(); kafkaTemplate.send("order.placed",
order.getId(), serialize(event)); } } // Consumer @KafkaListener(topics = "order.placed",
groupId = "inventory-service") public void handleOrderPlaced(ConsumerRecord<String, String>
record) { CloudEvent event = deserialize(record.value()); OrderPlacedData data =
parse(event.getData()); inventoryService.reserveStock(data.orderId(), data.lines()); }
```

Outbox Pattern – Spring Boot Implementation

```
// Outbox table CREATE TABLE outbox_events ( id UUID PRIMARY KEY, aggregate_type VARCHAR(100),
aggregate_id VARCHAR(100), event_type VARCHAR(200), payload JSONB, created_at TIMESTAMP,
published BOOLEAN DEFAULT false ); // Service – write order + outbox event in ONE transaction
@Service @Transactional public class OrderService { public void placeOrder(PlaceOrderCommand
cmd) { Order order = Order.create(cmd); orderRepository.save(order); // Same transaction –
guaranteed atomicity OutboxEvent outboxEvent = OutboxEvent.of( "Order", order.getId(),
"order.placed", toJson(order)); outboxRepository.save(outboxEvent); // NO Kafka call here –
relay handles publishing } } // Debezium CDC config (application.properties) //
connector.class=io.debezium.connector.postgresql.PostgresConnector //
table.include.list=public.outbox_events // transforms=outbox //
transforms.outbox.type=io.debezium.transforms.outbox.EventRouter
```

Interview Q&A;

Q1. What are the three types of events in event-driven architecture?

(1) Notification event: minimal payload, just signals something happened. Consumer calls back for details. (2) Event-carried state transfer: full state in payload, consumer needs no callback. (3) Domain event: meaningful domain occurrence, past tense, used for integration between bounded contexts. Martin Fowler's classification. Choose based on coupling vs. payload trade-off.

Q2. What is the difference between a domain event and an integration event?

Domain event: records a meaningful occurrence within a bounded context. Can be rich and context-specific. Used internally or passed to domain event handlers. Integration event: published to other bounded contexts via message broker. Must be stable, versioned, and follow Published Language principles. Domain events may be translated to integration events before publishing.

Q3. What is the outbox pattern and why is it needed?

Without the outbox pattern: save to DB and publish to Kafka are two separate operations. If the app crashes after DB save but before Kafka publish, the event is lost. Outbox pattern: write the event to an outbox table in the SAME database transaction as the business data. A relay (Debezium CDC) reads committed outbox records and publishes to Kafka. Guarantees events are published exactly as data is saved—no lost events.

Q4. What is a Schema Registry and why is it important?

A centralized service that stores Avro/Protobuf schema versions (Confluent Schema Registry). Producers register schemas before publishing. The Schema Registry enforces compatibility rules: BACKWARD (new schema reads old data), FORWARD (old schema reads new data), FULL (both). Prevents producers from

publishing schemas that break consumers. Schema IDs in Kafka messages enable versioning.

Q5. What is backward compatibility in event schema evolution?

A new schema version is backward compatible if consumers using the new schema can still read messages written with the old schema. In Avro: adding a field with a default value is backward compatible. Renaming or removing a field is NOT backward compatible. Schema Registry rejects backward-incompatible schema versions if BACKWARD compatibility is configured.

Q6. Explain the choreography pattern in event-driven architecture.

In choreography, services react to events published to a broker. No central coordinator. OrderService publishes OrderPlaced → InventoryService subscribes and reserves stock, publishes StockReserved → PaymentService subscribes and charges. Each service is decoupled—knows only the events, not other services. Pros: loose coupling, no SPOF. Cons: hard to trace end-to-end flow, no central view of saga state.

Q7. Explain the orchestration pattern in event-driven architecture.

A central Saga Orchestrator explicitly coordinates the saga. It sends ReserveStock command to Inventory, waits for reply, then sends ChargePayment to Payment. Maintains saga state machine (STARTED → STOCK_RESERVED → PAYMENT_CHARGED → COMPLETED). Pros: clear flow, easy to monitor, single point for compensation logic. Cons: orchestrator is a coordination bottleneck, more complex to implement.

Q8. What is an idempotent consumer and how do you implement it?

An idempotent consumer can safely process the same message multiple times (Kafka delivers at-least-once). Implementation: store processed event IDs in a deduplication table. Before processing: check if event ID already exists. If yes, skip. If no, process and insert ID atomically. The check + insert must be in the same transaction as the business operation.

Q9. What is the CloudEvents specification?

A CNCF standard for event metadata format. Defines envelope fields: specversion (1.0), id (unique), source (URI of event origin), type (reverse domain event name), time (ISO timestamp), datacontenttype (application/json), data (event payload). Ensures interoperability—any system understanding CloudEvents can process events from any producer.

Q10. How does Kafka guarantee ordering of events?

Kafka partitions guarantee ordering within a partition. Messages with the same partition key go to the same partition and are consumed in order. For order events: use orderId as partition key—all events for one order are in the same partition and processed in order. Events for different orders may be in different partitions.

Q11. What is event sourcing?

Instead of storing current state (UPDATE orders SET status = 'CONFIRMED'), store every state change as an immutable event (INSERT INTO order_events: OrderPlaced, OrderConfirmed, OrderShipped). Current state is derived by replaying all events. Provides full audit trail, temporal queries (state at any time), and natural event-driven integration. Downside: complex querying, snapshot needed for performance.

Q12. What is the difference between event sourcing and event-driven architecture?

EDA: services communicate via events on a message broker. Focuses on integration between services. Event Sourcing: a persistence mechanism where state changes are stored as events. Focuses on how state is stored. They complement each other but are independent concepts. You can use EDA without event sourcing (store current state, emit events). You can use event sourcing without EDA (no message broker needed).

Q13. How do you handle event ordering when consumers process events concurrently?

Partition-level ordering in Kafka: one consumer thread per partition. For strict ordering per aggregate: use aggregate ID as partition key—all events for that aggregate go to one partition and one consumer thread. Trade-off: fewer partitions = less parallelism. Optimistic locking on consumer side: check event version before applying.

Q14. What is a Dead Letter Queue and when is it used?

After exhausting retries (configurable backoff), failed messages are routed to a Dead Letter Topic/Queue. Prevents poison pill messages (malformed, always-failing) from blocking message processing. Operations team inspects DLQ, fixes the issue (code deploy, data fix), then replays the messages. Spring Kafka: configure DeadLetterPublishingRecoverer with exponential backoff.

Q15. Explain at-most-once, at-least-once, and exactly-once delivery semantics.

At-most-once: message may be lost but never duplicated (fire and forget). At-least-once: message is never lost but may be duplicated (requires idempotent consumers). Exactly-once: message delivered exactly once—hardest to achieve. Kafka supports exactly-once via idempotent producers + transactional API. In practice: design for at-least-once + idempotent consumers—simpler and more resilient.

Q16. How do you implement exactly-once semantics in Kafka?

Kafka's transactional API: producer.initTransactions(), producer.beginTransaction(), send messages + commit offsets in one transaction, producer.commitTransaction(). Requires: enable.idempotence=true, isolation.level=read_committed on consumers. Complex to implement. Most systems accept at-least-once + idempotent consumers instead.

Q17. What is event-driven vs request-driven architecture?

Request-driven: synchronous request/response (REST/gRPC). Caller waits for result. Simple, immediate feedback, easy to reason about. Couples availability: if downstream is down, caller fails. Event-driven: producer publishes event, continues. Consumer processes asynchronously. Decouples availability, higher throughput, but eventual consistency and harder debugging.

Q18. How do you trace a request across event-driven services?

Propagate a correlation/trace ID through all events. Include traceparent (W3C) or custom trace ID in event headers. Use distributed tracing: Micrometer Tracing auto-propagates trace context through Kafka message headers. Centralize logs in ELK/Loki. Filter by trace ID to see all events and logs for one business transaction.

Q19. What is the competing consumers pattern?

Multiple consumer instances in the same consumer group compete to process messages from a Kafka topic. Kafka assigns partitions to consumers: each partition is consumed by exactly one consumer in the group. Adding consumers (up to partition count) increases throughput linearly. Enables horizontal scaling of event processing.

Q20. What is event replay and when is it useful?

Re-consuming events from a Kafka topic from a specific offset (or beginning). Use cases: (1) New consumer service added—replay historical events to build state, (2) Bug fix—replay events with corrected logic, (3) New read model for CQRS—rebuild projection from event history. Requires Kafka's configurable retention period (set long enough for replay needs).

Q21. What is the Saga compensating transaction?

When a saga step fails mid-flow, compensating transactions semantically undo already-completed steps. Note: these are NOT database rollbacks (those already committed). They are new transactions that undo the effect. Example: if Payment fails after StockReserved, publish StockReleaseCommand to undo the reservation. Compensating transactions must be idempotent.

Q22. How do you handle schema evolution with Avro?

Add new fields with default values (backward compatible). Never remove fields that consumers depend on. Never rename fields (Avro matches by field name). To 'remove' a field: deprecate (mark in docs), wait for all consumers to stop using it, then remove in a future version. Schema Registry rejects incompatible schemas before they reach producers.

Q23. What are Kafka consumer groups?

A logical group of consumer instances that cooperatively consume a topic. Kafka distributes partitions among group members. Each message is processed by exactly one consumer in the group. Different consumer groups (services) each get a full copy of all messages. Example: order.placed topic consumed by both inventory-service group AND notification-service group independently.

Q24. How do you implement the outbox pattern with Debezium?

Debezium is a CDC (Change Data Capture) tool that reads the database transaction log (WAL in PostgreSQL). When a row is inserted into outbox_events table, Debezium captures the change and publishes to Kafka. No polling needed—CDC is near real-time. Debezium Outbox Event Router routes events to the correct Kafka topic based on aggregate_type field.

Q25. What is a Kafka partition key and how should you choose it?

Partition key determines which partition a message goes to: partition = hash(key) % numPartitions. All messages with the same key go to the same partition → ordering guaranteed within a key. Choose: (1) Aggregate ID for per-aggregate ordering (orderId, userId). (2) Tenant ID for per-tenant ordering in multi-tenant systems. Never use null key—distributes randomly, no ordering guarantees.

Q26. What is consumer lag and why does it matter?

Consumer lag: difference between latest message offset in a partition and the consumer's committed offset. High lag = consumer is falling behind producer. Indicates: slow consumer, insufficient consumers, or traffic spike. Monitor with Kafka metrics. Alert when lag exceeds threshold. Resolution: add consumers (up to partition count) or optimize consumer processing.

Q27. How do you handle poison pill messages in Kafka?

A poison pill is a message that always fails processing (malformed data, triggering a bug). Without handling: blocks all subsequent messages in the partition. Solution: retry with exponential backoff (Resilience4j + Spring Kafka). After max retries: route to Dead Letter Topic. Processing continues with next messages. Alert on DLQ messages for operations review.

Q28. What is event-driven CQRS?

Write side: processes commands, updates the write model, publishes domain events. Events flow to read model updaters via Kafka. Read side: subscribes to events, updates denormalized read models (Elasticsearch, Redis). No direct DB join queries needed for reads—read model is pre-computed and optimized. This is the natural combination of EDA with CQRS.

Q29. How do you test event-driven services?

Unit: mock event publisher, verify events are constructed correctly. Integration: Testcontainers with real Kafka broker. Publish event, verify consumer processes it, check DB state. Contract testing (Pact): verify event schema matches consumer expectations. E2E: full flow test using Docker Compose with all services and Kafka.

Q30. What is event enrichment?

Adding data to an event that the consumer needs but the producer doesn't naturally have. Example: PaymentService needs customer email for receipt. Options: (1) Include in integration event (state transfer).

(2) Consumer fetches additional data via API call (notification event + callback). (3) Event enrichment service reads event, augments with data from other services, re-publishes enriched event.

Q31. What is temporal decoupling in event-driven architecture?

Producer and consumer don't need to be running at the same time. Producer publishes event when it has data; consumer processes whenever it's running. If consumer is down: events accumulate in Kafka (durable log). Consumer catches up when restarted. This is fundamentally different from synchronous REST/gRPC where both must be up simultaneously.

Q32. How does Kafka ensure durability of events?

Messages are written to disk (append-only log) with configurable replication factor (typically 3). Producer uses acks=all: message only confirmed after all replicas receive it. min.insync.replicas=2: at least 2 replicas must acknowledge. Retention period: configurable (days, bytes). Default 7 days. Compacted topics: retain only last message per key indefinitely.

Q33. What is the event sourcing projection?

A view of the current state computed by replaying the event log. Example: to find current order status, replay all OrderPlaced, OrderConfirmed, OrderShipped events. Projections can be materialized and cached (snapshots) for performance. Multiple different projections can be built from the same event stream for different query needs.

Q34. Describe a Spring Boot Kafka producer configuration.

spring.kafka.bootstrap-servers=kafka:9092
spring.kafka.producer.key-serializer=org.apache.kafka.common.serialization.StringSerializer
spring.kafka.producer.value-serializer=io.confluent.kafka.serializers.KafkaAvroSerializer
spring.kafka.producer.acks=all spring.kafka.producer.enable-idempotence=true
spring.kafka.producer.retries=3 schema.registry.url=http://schema-registry:8081

Q35. What are the trade-offs of event-driven vs synchronous communication?

Event-driven pros: loose coupling, temporal decoupling, independent scaling, natural audit trail. Event-driven cons: eventual consistency, harder debugging (distributed trace needed), out-of-order event handling, idempotency required, schema management overhead. Synchronous pros: simple, immediate feedback, easy error handling. Synchronous cons: tight coupling, cascading failures, shared availability. Most systems use both: sync for reads, async for writes/side effects.

Q36. What is saga orchestration with Eventuate Tram?

Eventuate Tram is a Java framework for transactional messaging and orchestration sagas. Provides: (1) @Transactional outbox pattern built-in, (2) SagaManager that persists saga state and sends commands. SagaDefinition.step().invokeParticipant(inventoryService::reserve)
.withCompensation(inventoryService::release). Eliminates boilerplate for outbox, saga state machine, and compensation handling.

Q37. How do you ensure event ordering for a multi-partition topic?

Use a consistent partition key (aggregate ID) so all events for one aggregate go to the same partition. Within a consumer group, each partition is processed by one thread—ordering preserved. For global ordering: use one partition (limits throughput). For relative ordering (event A before event B per entity): partition by entity ID.

Q38. What is the event-carried state transfer anti-pattern?

Including the full entity state in every event is an anti-pattern when the state is large, changes frequently, or contains sensitive data. Better: include only the changed fields plus the aggregate ID. Consumers that need full state: call the producer's API (notification event pattern). Balance between bandwidth efficiency and

callback overhead based on use case.

Q39. How do you handle multiple consumers needing different data from the same event?

Kafka's consumer group model: each service subscribes independently to the same topic. If different consumers need different data enrichment: (1) include all potentially needed data in event, (2) each consumer fetches additional data it needs via API, (3) use an event enrichment service that publishes enriched versions of events to separate topics.

Q40. What is the difference between Kafka topics and RabbitMQ queues?

Kafka topic: persistent log. Messages retained for configurable period. Multiple consumer groups each get all messages. Supports replay. Consumers pull. RabbitMQ queue: message deleted after consumption. One consumer gets each message. No native replay. Push-based delivery. Kafka is better for event streaming; RabbitMQ for simple task queuing.

Q41. What is backpressure in event-driven systems?

When consumers can't keep up with producer throughput. Kafka consumers naturally apply backpressure by controlling the poll rate and max.poll.records. If consumer is slow: increase consumer count (up to partition count), optimize processing, or reduce max.poll.records. Reactive streams (Project Reactor) support backpressure propagation in stream processing.

Q42. How do you monitor an event-driven system?

Key metrics: consumer lag per partition (alert when high), messages produced/consumed per second, processing latency per consumer, Dead Letter Queue size (alert when non-zero), Schema Registry health. Tools: Kafka JMX metrics → Prometheus → Grafana. Distributed tracing: Micrometer Tracing propagates trace IDs through Kafka message headers.

Q43. What is a compacted Kafka topic and when is it used?

A compacted topic retains only the latest message per key—old versions are garbage collected. Effectively a key-value store. Use cases: (1) Configuration updates (service always gets latest config on startup), (2) Event sourcing snapshots, (3) CDC table replication (latest row state per primary key). Not suitable for event history—use regular topics with retention for full history.